```
"""
render_pipeline.py — Layered render pipeline for Zenith-Pygame.

Replaces the hardcoded draw-order in each biome's game_draw() with a
named-layer system. Entities register draw callbacks on a specific layer;
the pipeline flushes them in order with optional Y-sorting within a layer.

Usage:
    pipeline = RenderPipeline()
    pipeline.submit("entities", lambda s: player.draw(s, cx, cy),
                    sort_key=player.rect.centery)
    pipeline.submit("hud", lambda s: draw_game_hud(s))
    pipeline.flush(screen)   # draws all layers in order, clears queues
"""

from collections import defaultdict

class RenderPipeline:
    """Layered render pipeline with per-layer Y-sorting."""

    LAYERS = [
        "floor",        # 0 — tile fills
        "walls",        # 1 — wall segments + shadows
        "decals",       # 2 — blood, scorch marks
        "items",        # 3 — pickups, desk items
        "doors",        # 4 — doors, breach markers
        "shadows",      # 5 — entity drop-shadows (drawn below entities)
        "entities",     # 6 — player, enemies, NPCs
        "projectiles",  # 7 — bullets, thrown bottles
        "effects",      # 8 — particles, casings, fire zones
        "floating",     # 9 — floating text popups
        "hud",          # 10 — minimap, ammo, health, objectives, crosshair
        "overlay",      # 11 — muzzle flash, screen flash, transition
        "postprocess",  # 12 — grain, scanlines, vignette, brightness
    ]

    def __init__(self):
        self._drawables = defaultdict(list)
        self._layer_index = {name: i for i, name in enumerate(self.LAYERS)}

    def submit(self, layer, draw_func, sort_key=None):
        """Register a draw callback for this frame.

        Args:
            layer: One of RenderPipeline.LAYERS.
            draw_func: callable(surface) — draws onto the screen.
            sort_key: Optional numeric key for Y-sorting within the layer.
                      Lower values are drawn first (behind).
        """
        if layer not in self._layer_index:
            raise ValueError(
                f"Unknown render layer '{layer}'. "
                f"Valid layers: {', '.join(self.LAYERS)}"
            )
        self._drawables[layer].append((sort_key or 0, draw_func))

    def flush(self, surface):
        """Draw all submitted callbacks in layer order, then clear queues.

        Within each layer, items are sorted by sort_key (Y-sorting).
        """
        for layer_name in self.LAYERS:
            items = self._drawables[layer_name]
            if not items:
                continue
            # Only sort if there's more than one item and at least one
            # has a non-zero sort key (avoid unnecessary work for HUD etc.)
            if len(items) > 1:
                items.sort(key=lambda x: x[0])
            for _, draw_func in items:
                draw_func(surface)
            items.clear()

    def clear(self):
        """Discard all pending draw calls without rendering."""
        for layer_name in self.LAYERS:
            self._drawables[layer_name].clear()

    @property
    def pending_count(self):
        """Number of draw calls waiting to be flushed (debug use)."""
        return sum(len(v) for v in self._drawables.values())
```